

Universidad Nacional Autónoma de México



Facultad de Ingeniería

Alumno:

Lara Hernandez Angel Husiel - 320060829

Arquitectura Cliente-Servidor

Grupo: 02 - Semestre: 2027-1

Tarea 4:

Sockets Internet y cola de espera.

Profesor:

MTRO. CARLOS ALBERTO ROMAN ZAMITIZ

Fecha de realización:

8 de octubre del 2026

Índice

1 Investigacion sobre los comandos y la funcion sk_acceptq_is_full() 1
1.1 Funcionamiento del programa: 1
1.2 Función sk_acceptq_is_full(): 1
1.3 Investigacion de cat /proc/sys/net/ipv4/tcp_syncookies: 1
1.4 Investifacion cat /proc/sys/net/ipv4/tcp_syn_retries: 1

2 Codigo e implementacion: 2
2.1 Configuracion de l terminal: 2
2.2 Compilación del servidor: 3
2.3 Servidor y primera conexión: 3
2.4 Ejecución de los cuatro clientes: 4
2.5 Consulta de las conexiones con ss: 5
2.6 Restauración de los parámetros: 7

3 Conclusión 8

4 Referencias 8

1. Investigacion sobre los comandos y la funcion sk_acceptq_is_full()

1.1. Funcionamiento del programa:

1.2. Función sk_acceptq_is_full():

Primero que nada, en este caso el codigo de esta funcion, esta en : `include/net/sock.h`, esta es una función interna del núcleo que consulta la cola Accept del socket.

Para la parte de `sk_ack_backlog` lleva la cantidad de conexiones pendientes y `sk_max_ack_backlog` contiene el límite de la cola. La comprobación se realiza de la siguiente forma:

Listing 1: Comparación utilizada para revisar la cola Accept.

```
static inline bool sk_acceptq_is_full(const struct sock *sk)
{
    return READ_ONCE(sk->sk_ack_backlog) >
           READ_ONCE(sk->sk_max_ack_backlog);
}
```

Ahora bien, en el cso de la funcion utiliza `>`, esto basado en backlog 1, la comparación $1 > 1$ es falsa y puede entrar otra conexión pendiente; con dos, $2 > 1$ es verdadera.

Por lo tanto, en esta prueba el primer cliente es atendido y los siguientes dos esperan, pero ya para el cuarto se ve el desbordamiento.

1.3. Investigacion de cat /proc/sys/net/ipv4/tcp_syncookies:

```
cat /proc/sys/net/ipv4/tcp_syncookies
```

Para el caso de este comando, `cat` muestra el valor actual de un parámetro TCP expuesto por Linux en `/proc/sys/net/ipv4/`. La consulta no cambia la configuración. En mi linux se obtuvo 1 antes de realizar los ajustes.

Ahora bien, `tcp_syncookies` controla las SYN cookies, que sirven como protección cuando se desborda la cola SYN. El valor 0 las desactiva, 1 permite utilizarlas ante ese desbordamiento y 2 las genera de forma incondicional.

1.4. Investifacion cat /proc/sys/net/ipv4/tcp_syn_retries:

```
cat /proc/sys/net/ipv4/tcp_syn_retries
```

Por otra parte, este comando consulta el límite de retransmisiones del SYN durante un intento de conexión activa, en el caso de mi linux se obtuvo 6, esto es con relacion a los intentos que hace el cliente cuando la conexión no se completa; no significa que el servidor admita seis clientes.

Lo que hace el 1 es que reduce ese límite. Este parámetro es distinto de `tcp_synack_retries`, que corresponde a las retransmisiones SYN+ACK del servidor. El cambio tampoco modifica el backlog.

2. Codigo e implementacion:

Al iniciar se crea el socket por medio de `socket()`, después se coloca la dirección y el puerto en la estructura `sockaddr_in`. La llamada `bind()` asocia el socket con las direcciones locales del servidor, esto debido a que se utiliza `INADDR_ANY`. Por último se llama a `listen()` y el proceso espera una conexión en `accept()`.

Cuando el primer cliente es aceptado, el programa entra a un segundo ciclo que utiliza `recv()`. Lo que sucede aquí es que el servidor permanece ocupado con ese cliente mientras no se desconecte. Si no se envían datos, `recv()` queda bloqueada, por lo que el proceso no regresa a ejecutar `accept()` para los siguientes clientes. No obstante, el sistema operativo continúa procesando las solicitudes de conexión y administra las colas TCP.

La cola SYN contiene solicitudes cuyo establecimiento todavía no se ha completado desde el lado del servidor. En una conexión normal, el cliente envía SYN, el servidor contesta SYN+ACK y el cliente responde ACK. Cuando el servidor procesa este último paso, la conexión puede pasar a la cola Accept.

La cola Accept contiene conexiones establecidas que todavía no han sido retiradas por el programa mediante `accept()`. Esto es importante, porque el mensaje de conexión exitosa de `nc` corresponde al establecimiento TCP visto por el cliente, y por sí solo no demuestra que el servidor ya esté atendiendo a ese cliente.

2.1. Configuracion de l terminal:

En este caso, `sudo` permite ejecutar el cambio con permisos de administrador, mientras que `sysctl -w` escribe el valor del parámetro del sistema en ejecución. El primer comando desactiva las SYN cookies y el segundo reduce los reintentos SYN. Los cambios se realizan para observar el comportamiento del laboratorio y al terminar se deben regresar a los valores guardados, sin modificar los archivos de configuración permanente.

```
sudo sysctl -w net.ipv4.tcp_syncookies=0
sudo sysctl -w net.ipv4.tcp_syn_retries=1
```

Ahora bien, el compilador regresó a la línea de comandos sin mostrar errores. Después se consultó cada parámetro por separado, donde se obtuvieron los valores originales 1 y 6:

```
cat /proc/sys/net/ipv4/tcp_syncookies
1
cat /proc/sys/net/ipv4/tcp_syn_retries
6
```

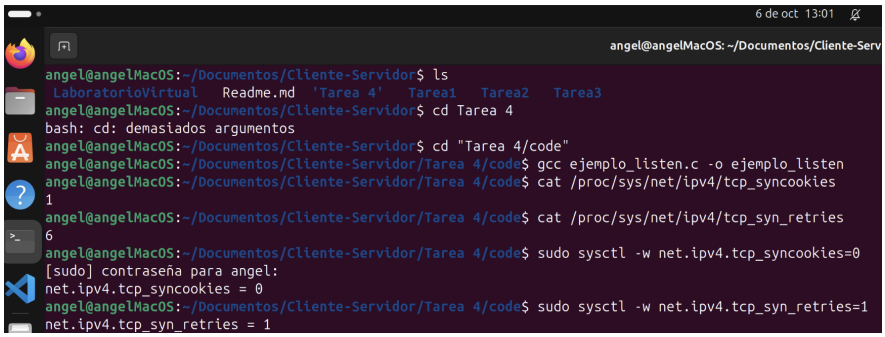


Figura 1: Compilación, consulta de parámetros, aplicación de los ajustes y restauración de los valores originales.

Con los comandos sudo, puesto correspondientemente, ahora el `tcp_syncookies` tiene un valor de 0 y `tcp_syn_retries` de 1. Esto permite observar el comportamiento del servidor cuando se desborda la cola SYN, además de reducir el tiempo de espera del cuarto cliente.

2.2. Compilación del servidor:

Desde la raíz del repositorio se ejecutan los siguientes comandos dentro de Linux:

```
cd "Tarea 4/code"
gcc ejemplo_listen.c -o ejemplo_listen
./ejemplo_listen 4898
```

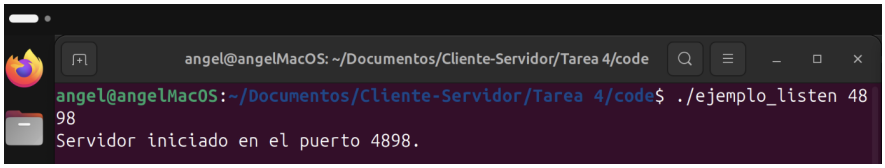


Figura 2: Compilación y ejecución del servidor en el puerto 4898.

2.3. Servidor y primera conexión:

En la terminal de la izquierda se ejecutó `./ejemplo_listen 4898`, donde el programa mostró que había iniciado en el puerto 4898 y que estaba esperando en `accept()`. Por otra parte, en la terminal de la derecha se ejecutó `nc -v 127.0.0.1 4898`, que mostró el mensaje de conexión exitosa.

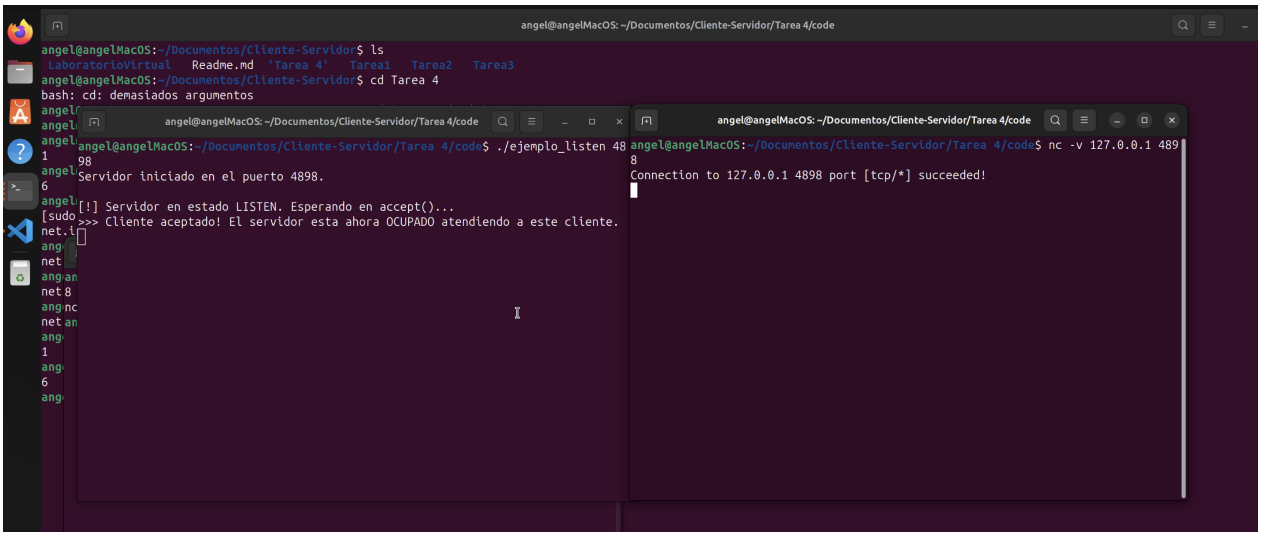


Figura 3: Servidor en el puerto 4898 y primer cliente conectado.

Lo que vemos aquí es que el servidor imprime el cliente fue aceptado

y señaló que estaba ocupado atendiendo a ese cliente. Esto permite comprobar que la primera conexión sí fue retirada de la cola por medio de `accept()`. Como el cliente permaneció abierto sin enviar datos, el servidor continuó esperando en `recv()`.

2.4. Ejecución de los cuatro clientes:

Para los clientes se abren cuatro terminales, ejecutando el siguiente comando en cada una, de forma consecutiva:

```
nc -v 127.0.0.1 4898
```

El parámetro `-v` muestra información del intento de conexión. El primer cliente debe mantenerse abierto para que el servidor permanezca ocupado. Después se ejecutan los clientes segundo, tercero y cuarto, manteniendo las terminales visibles para comparar los resultados.

Ahora bien, se ve el servidor en la parte superior izquierda y el primer cliente en la parte superior derecha. Los clientes segundo y tercero se encuentran en las terminales inferiores, donde ambos mostraron que la conexión TCP fue exitosa. No obstante, el servidor solamente mostró una conexión aceptada, por lo que esos dos clientes todavía estaban esperando su turno de atención.

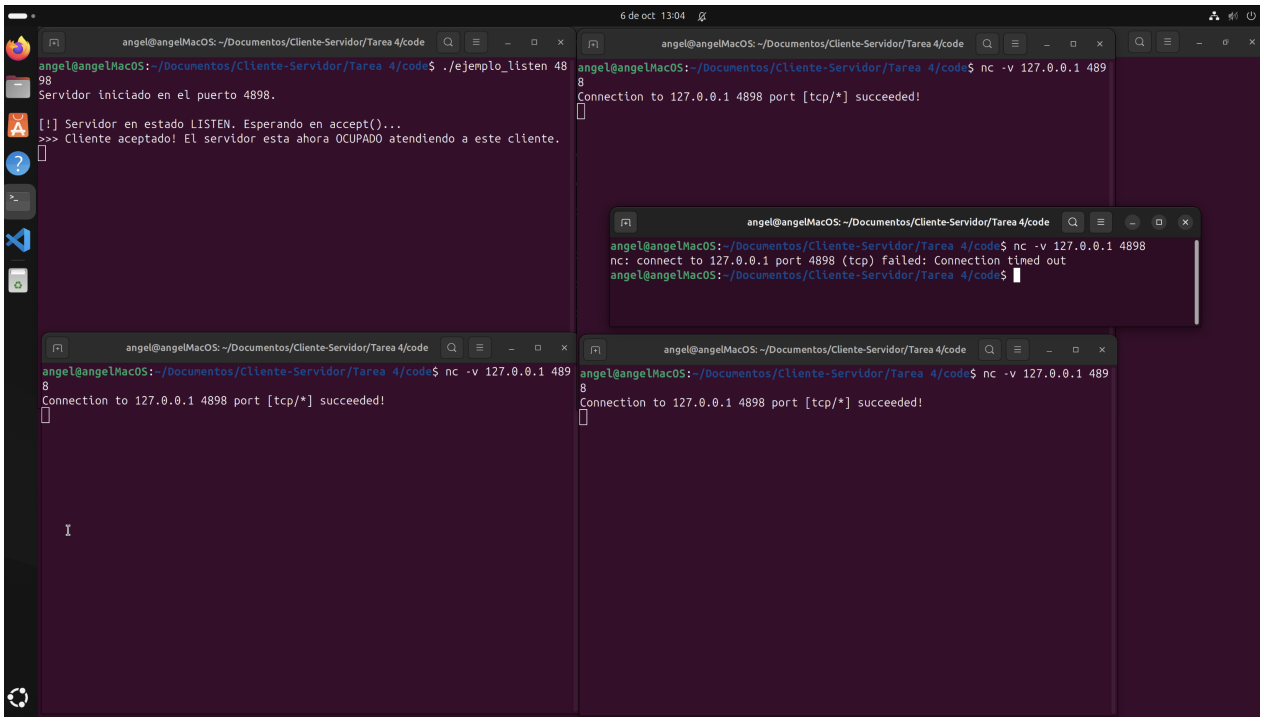


Figura 4: Servidor y cuatro clientes: tres conexiones exitosas y un intento que terminó por tiempo de espera.

Por otra parte, la terminal pequeña de la derecha corresponde al cuarto cliente. En este caso, el comando terminó con `Connection timed out`, lo que significa que no pudo completar la conexión dentro del tiempo de espera. No se observó un rechazo inmediato; el resultado que se obtuvo fue el agotamiento del tiempo de conexión.

2.5. Consulta de las conexiones con ss:

En una sexta terminal se ejecuta:

```
ss -atnp | grep :4898
watch -n 1 "ss -atn 'sport = :4898'"
```

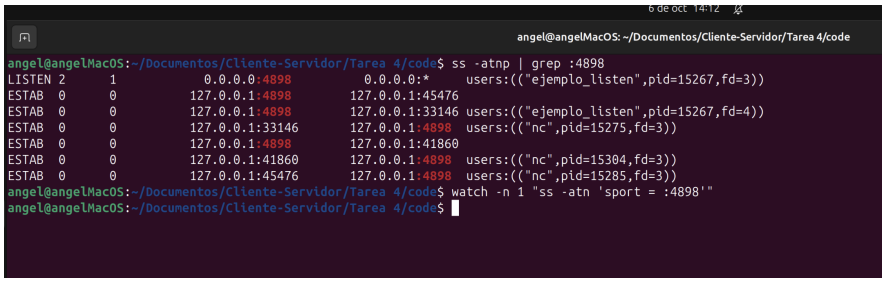


Figura 5: Vista de la finalizacion del comando, una vez cerrado els ervidor.

El primer comando muestra los sockets TCP relacionados con el puerto, junto con la información de proceso que esté disponible para el usuario. El segundo actualiza cada segundo los sockets cuyo puerto local es 4898, esto para ver el estado del servidor.

En la fila `LISTEN`, la columna `Recv-Q` indica cuántas conexiones están esperando a `accept()`, mientras que `Send-Q` muestra el backlog configurado. Por lo tanto, una fila `LISTEN 2 1` permite observar dos conexiones pendientes con backlog 1. En las filas `ESTAB`, esas mismas columnas representan bytes, por lo que no se interpretan como número de clientes.

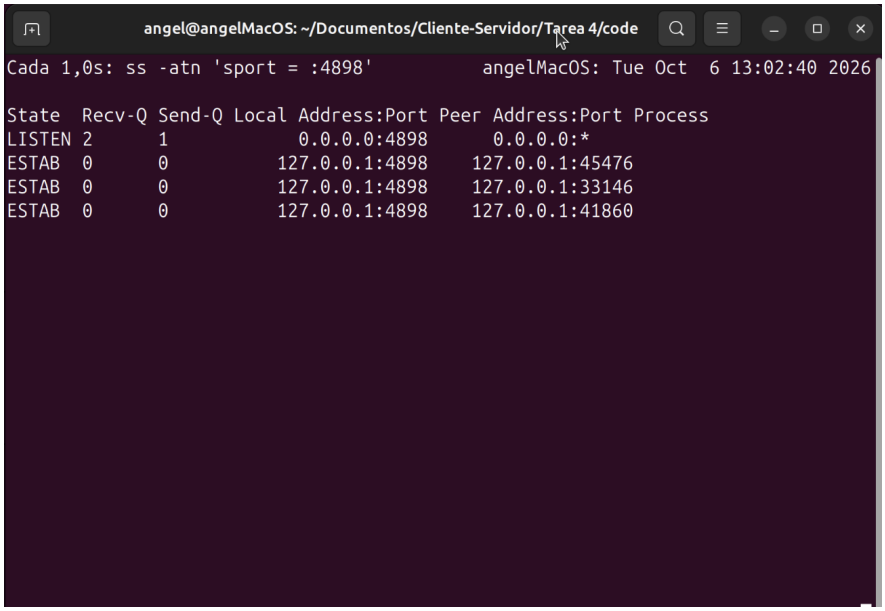


Figura 6: Monitor del puerto 4898: dos conexiones pendientes con backlog 1.

Para el caso de esta consulta, lo que vemos en la fila `LISTEN` es un valor de 2 en `Recv-Q` y de 1 en `Send-Q`. Esto indica que había dos conexiones esperando a que el programa volviera a ejecutar `accept()`, aunque el backlog del código tiene valor 1.

Además de eso, aparecen tres filas `ESTAB` del lado del servidor, relacionadas con los puertos de cliente 45476, 33146 y 41860. La conexión que ya había sido aceptada correspondió al puerto 33146, mientras que las otras dos permanecieron en la cola. En estas filas, los valores 0 de `Recv-Q` y `Send-Q` representan bytes; no significan que la cola Accept estuviera vacía.

Vista final:

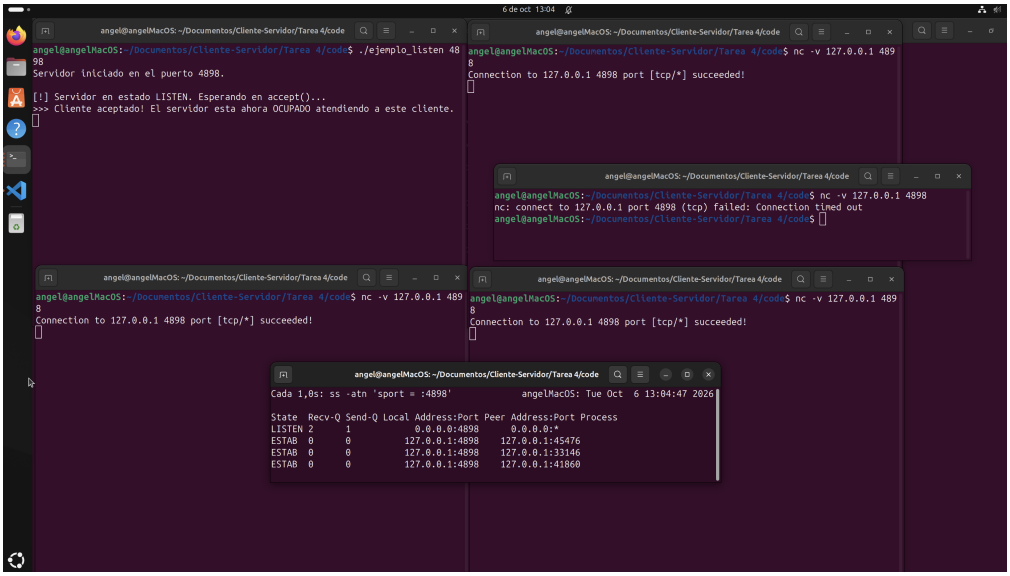


Figura 7: Vista conjunta del servidor, los cuatro clientes y el monitor de conexiones.

2.6. Restauración de los parámetros:

Al terminar los intentos de conexión se regresaron los valores originales, escribiendo cada comando por separado:

```
sudo sysctl -w net.ipv4.tcp_syncookies=1
sudo sysctl -w net.ipv4.tcp_syn_retries=6
cat /proc/sys/net/ipv4/tcp_syncookies
1
cat /proc/sys/net/ipv4/tcp_syn_retries
6
```

Como se ve en la parte final de la figura, la consulta devolvió 1 y 6. Por lo tanto, los cambios temporales quedaron restaurados y no se modificó la configuración permanente de la máquina virtual.

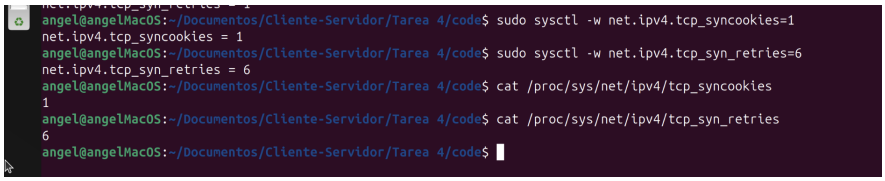


Figura 8: Restauracion de los parametros normales.

3. Conclusión

Para el caso de esta actividad se comprobó que el servidor atiende un cliente a la vez debido al ciclo con `recv()`, mientras que Linux mantiene otras conexiones esperando en la cola Accept. Aunque se configuró backlog 1, la consulta mostró dos conexiones pendientes, lo que coincide con la comparación estricta de `sk_acceptq_is_full()`.

Ahora bien, los tres primeros clientes establecieron la conexión TCP, pero únicamente el primero fue atendido por el programa. El cuarto terminó con `Connection timed out`. Esto permite distinguir entre conectarse, quedar en espera y ser aceptado por la aplicación, además de explicar por qué fueron necesarios cuatro clientes para observar el desbordamiento en la máquina virtual utilizada.

4. Referencias

1. Linux Kernel. Documentación de parámetros IP sysctl: `tcp_syncookies`, `tcp_syn_retries` y `tcp_abort_on_overflow`.
   https://docs.kernel.org/networking/ip-sysctl.html
2. OpenAnolis. TCP SYN Queue and Accept Queue Overflow Explained (2022).
   https://www.alibabacloud.com/blog/tcp-syn-queue-and-accept-queue-overflow-explained_599203
3. Linux man-pages. Manual de TCP, interfaces de `/proc` y parámetros de conexión.
   https://man7.org/linux/man-pages/man7/tcp.7.html